Библиотеки работы с данными NumPy

NumPy: начало работы ... 3
Операции с использованием NumPy ... 3
NumPy — Окружающая среда ... 3
Импорт модуля numpy ... 3
Наиболее важные атрибуты объектов ndarray: ... 4
Создание массивов ... 4
Печать массивов ... 7
NumPy: базовые операции над массивами ... 7
Индексы, срезы, итерации ... 9
Манипуляции с формой ... 12
Объединение массивов ... 14
Разбиение массива ... 15
Копии и представления ... 15
Представление или поверхностная копия ... 16
Глубокая копия ... 17
Математика многочленов ... 17
Статистика ... 18
NumPy: случайные числа ... 19
Инициализация генератора случайных чисел ... 19
Другой способ – numpy.random ... 21
Создание массивов ... 21
Выбор и перемешивание ... 23
Операторы сравнения и тестирование значений ... 24
Выбор элементов массива и манипуляция с ними ... 26
Практическое задание ... 29
Ресурсы: ... 30

NumPy: начало работы

NumPy – базовая библиотека с открытым исходным кодом для вычислений в среде Python. Состоит из объектов многомерного массива и набора процедур для обработки массива. Предлагает поддержку многомерных массивов, матриц и эффективных функций для работы с этими типами данных. Быстродействие кода Python с использованием NumPy в 50 раз быстрее кода на “чистом” Python.
Основным объектом NumPy является однородный многомерный массив (в numpy называется numpy.ndarray). Это многомерный массив элементов (обычно вложенная последовательность чисел), одного типа.

Операции с использованием NumPy

Используя NumPy, разработчик может выполнять следующие операции:

- Математические и логические операции над массивами.
- Преобразования Фурье и процедуры для манипуляции с формой.
- Операции, связанные с линейной алгеброй. NumPy имеет встроенные функции для линейной алгебры и генерации случайных чисел.

NumPy — Окружающая среда

Стандартный дистрибутив Python не поставляется в комплекте с модулем NumPy. Возможна установка NumPy с помощью популярного установщика пакетов Python, pip. **pip install numpy**
Лучший способ включить NumPy – это использовать устанавливаемый двоичный пакет, соответствующий операционной системе ПК. Эти двоичные файлы содержат полный стек SciPy (включая NumPy, SciPy, matplotlib, IPython, SymPy и пакеты носа[*] вместе с ядром Python).

Импорт модуля numpy

Есть несколько путей импорта. Стандартный метод это — использовать простое выражение:

```
>>> import numpy
```

Для большого количества вызовов функций numpy возможно сделать это так:

```
>>> import numpy as np
```

Такое выражение позволяет получать доступ к numpy-объектам, используя np.X вместо numpy.X.
Также можно импортировать numpy прямо в используемое пространство имен, чтобы вообще не использовать функции через точку, а вызывать их напрямую:

```
>>> from numpy import *
```

Однако этот вариант не приветствуется в программировании на python, так как исключает некоторые полезные структуры, предоставляемые модулем. До конца

[*] носа? издержки перевода.

рабочей тетради мы будем использовать второй вариант импорта (import numpy as np).

Наиболее важные атрибуты объектов ndarray:

- ndarray.ndim – число измерений (чаще их называют "оси") массива.
- ndarray.shape – размеры массива, его форма. Это кортеж натуральных чисел, показывающий длину массива по каждой оси. Для матрицы из n строк и m столбов, shape будет (n,m). Число элементов кортежа shape равно ndim.
- ndarray.size – количество элементов массива. Равно произведению всех элементов атрибута shape.
- ndarray.dtype – объект, описывающий тип элементов массива. Можно определить dtype, используя стандартные типы данных Python. NumPy здесь предоставляет целый букет возможностей, как встроенных, например: bool_, character, int8, int16, int32, int64, float8, float16, float32, float64, complex64, object_, так и возможность определить собственные типы данных, в том числе и составные.
- ndarray.itemsize – размер каждого элемента массива в байтах.
- ndarray.data – буфер, содержащий фактические элементы массива. Атрибут редко используется, так как обращаться к элементам массива проще всего с помощью индексов.

Создание массивов

В NumPy существует много способов создать массив. Один из наиболее простых - создать массив из обычных списков или кортежей Python, используя функцию **numpy.array().**
array - функция, создающая объект типа ndarray

```
>>> import numpy as np
>>> a=np.array([1, 2, 3])
>>>a
```
Выведет array([1, 2, 3]).

```
>>>type(a)
```
Результат :
```
<class 'numpy.ndarray'>
```

Функция **array**() трансформирует вложенные последовательности в многомерные массивы. Тип элементов массива зависит от типа элементов исходной последовательности.

```
>>>b = np.array([[1.5, 2, 3], [4, 5, 6]])
>>>b
```
Результат:
```
array([[ 1.5,  2. ,  3. ],
       [ 4. ,  5. ,  6. ]])
```

Можно также переопределить тип в момент создания:

```
>>> b = np.array([[1.5, 2, 3], [4, 5, 6]], dtype=np.complex)
>>>b
```

Результат выполнения программного кода:

```
array([[ 1.5+0.j,  2.0+0.j,  3.0+0.j],
       [ 4.0+0.j,  5.0+0.j,  6.0+0.j]])
```

Функция array() не единственная функция для создания массивов. Нередко элементы массива вначале неизвестны, а массив, в котором они будут храниться, уже нужен. Существует несколько функций для того, чтобы создавать массивы с каким-то исходным содержимым (по умолчанию тип создаваемого массива – float64).

Функция **zeros**() создает массив из нулей, а функция **ones**() — массив из единиц. Обе функции принимают кортеж с размерами, и аргумент dtype:

```
>>>np.zeros((3, 5))
```

Результат:

```
array([[ 0.,  0.,  0.,  0.,  0.],
       [ 0.,  0.,  0.,  0.,  0.],
       [ 0.,  0.,  0.,  0.,  0.]])
```

```
>>>np.ones((2, 2, 2))
```

Результат:

```
array([[[ 1.,  1.],
        [ 1.,  1.]],
       [[ 1.,  1.],
        [ 1.,  1.]]])
```

Функция **eye**() создаёт единичную матрицу (двумерный массив)

```
>>>np.eye(5)
```

Результат:

```
array([[ 1.,  0.,  0.,  0.,  0.],
       [ 0.,  1.,  0.,  0.,  0.],
       [ 0.,  0.,  1.,  0.,  0.],
       [ 0.,  0.,  0.,  1.,  0.],
       [ 0.,  0.,  0.,  0.,  1.]])
```

Функция **empty**() создает массив без его заполнения. Исходное содержимое случайно и зависит от состояния памяти на момент создания массива (то есть от того мусора, что в ней хранится):

```
>>> np.empty((3, 3))
```

Результат:

```
array([[  6.93920488e-310,   6.93920488e-310,   6.93920149e-310],
       [  6.93920058e-310,   6.93920058e-310,   6.93920058e-310],
       [  6.93920359e-310,   0.00000000e+000,   6.93920501e-310]])
```

```
>>>np.empty((3, 3))
Результат:
        array([[  6.93920488e-310,   6.93920488e-310,   6.93920147e-310],
        [  6.93920149e-310,   6.93920146e-310,   6.93920359e-310],
        [  6.93920359e-310,   0.00000000e+000,   3.95252517e-322]])
```

Для создания последовательностей чисел, в NumPy имеется функция **arange**(), аналогичная встроенной в Python range(), только вместо списков она возвращает массивы, и принимает не только целые значения:

```
>>>np.arange(10, 30, 5)
Результат:
        array([10, 15, 20, 25])

>>>np.arange(0, 1, 0.1)
Результат:
        array([ 0. ,  0.1,  0.2,  0.3,  0.4,  0.5,  0.6,  0.7,  0.8,  0.9])
```

Причем, при использовании **arange()** с аргументами типа float, неизвестно, сколько элементов будет получено (из-за ограничения точности чисел с плавающей запятой). Поэтому, в таких случаях рекомендуется использовать функцию **linspace(),** которая вместо шага в качестве одного из аргументов принимает число, равное количеству нужных элементов:

```
>>>np.linspace(0, 2, 9)  # 9 чисел от 0 до 2 включительно
Результат:
        array([  0. ,  0.25,  0.5 ,  0.75, 1. , 1.25, 1.5 , 1.75, 2. ])
```

fromfunction(): применяет функцию ко всем комбинациям индексов

```
>>>def f1(i, j):
...     return 3 * i + j
...
>>>np.fromfunction(f1, (3, 4))
Результат:
        array([[ 0.,  1.,  2.,  3.],
                [ 3.,  4.,  5.,  6.],
                [ 6.,  7.,  8.,  9.]])

>>>np.fromfunction(f1, (3, 3))
Результат:
        array([[ 0.,  1.,  2.],
                [ 3.,  4.,  5.],
                [ 6.,  7.,  8.]])
```

Печать массивов

Если массив слишком большой, чтобы его печатать полностью, NumPy автоматически скрывает центральную часть массива и выводит только его начало и конец.

```
>>>print(np.arange(0, 3000, 1))
```

Результат:

```
[   0    1    2 ..., 2997 2998 2999]
```

Если есть необходимость увидеть весь массив, используйте функцию **numpy.set_printoptions:**

np.set_printoptions(threshold=np.nan)

С помощью данной функции возможно настроить печать массивов индивидуально. Функция numpy.set_printoptions принимает несколько аргументов: precision - количество отображаемых цифр после запятой (по умолчанию 8). threshold - количество элементов в массиве, вызывающее обрезание элементов (по умолчанию 1000).

edgeitems – количество элементов в начале и в конце каждой размерности массива (по умолчанию 3).

linewidth – количество символов в строке, после которых осуществляется перенос (по умолчанию 75).

suppress – если True, не печатает маленькие значения в scientific notation (по умолчанию False).

nanstr – строковое представление NaN (по умолчанию 'nan').

infstr – строковое представление inf (по умолчанию 'inf').

formatter – позволяет более тонко управлять печатью массивов.

NumPy: базовые операции над массивами

Математические операции над массивами выполняются поэлементно. Создается новый массив, который заполняется результатами действия оператора. >>> import numpy as np

```
>>> a = np.array([20, 30, 40, 50])
>>> b = np.arange(4)
>>> a + b
```

Результат:

```
array([20, 31, 42, 53])
```

```
>>> a - b
```

Результат:

```
array([20, 29, 38, 47])
```

```
>>>a * b
```

Результат:

```
array([  0,  30,  80, 150])
```

```
>>>a / b   # При делении на 0 возвращается inf (бесконечность)
Результат:
        array([        inf,  30.        ,  20.        ,  16.66666667])
<string>:1: RuntimeWarning: divide by zero encountered in true_divide

>>>a ** b
Результат:
        array([     1,     30,   1600, 125000])

>>>a % b    # При взятии остатка от деления на 0 возвращается 0
Результат:
<string>:1: RuntimeWarning: divide by zero encountered in remainder
array([0, 0, 0, 2])
```

Для этого, естественно, массивы должны быть одинаковых размеров.

```
>>> c = np.array([[1, 2, 3], [4, 5, 6]])
>>> d = np.array([[1, 2], [3, 4], [5, 6]])
>>> c + d
Результат:
Traceback (most recent call last):
  File "<input>", line 1, in <module>
ValueError: operands could not be broadcast together with shapes (2,3) (3,2)
```

Также можно производить математические операции между массивом и числом. В этом случае к каждому элементу прибавляется это число.

```
>>>a + 1
Результат:
        array([21, 31, 41, 51])

>>>a ** 3
Результат:
        array([  8000,  27000,  64000, 125000])

>>>a< 35   # И фильтрацию можно проводить
Результат:
        array([ True,  True, False, False], dtype=bool)
```

NumPy также предоставляет множество математических операций для обработки массивов:

```
>>>np.cos(a)
Результат:
        array([ 0.40808206,  0.15425145, -0.66693806,  0.96496603])
```

```
>>>np.arctan(a)
Результат:
array([ 1.52083793,  1.53747533,  1.54580153,  1.55079899])

>>>np.sinh(a)
Результат:
array([  2.42582598e+08,   5.34323729e+12,   1.17692633e+17, 2.59235276e+21])
```

Многие унарные операции, такие как, например, вычисление суммы всех элементов массива, представлены также и в виде методов класса ndarray.

```
>>>a = np.array([[1, 2, 3], [4, 5, 6]])
>>>np.sum(a)
Результат:   21

>>>a.sum()
Результат:   21

>>>a.min()
Результат:    1

>>>a.max()
Результат:   6
```

По умолчанию эти операции применяются к массиву, как если бы он был списком чисел, независимо от его формы. Однако, указав параметр axis, можно применить операцию для указанной оси массива:

```
>>>a.min(axis=0)  # Наименьшее число в каждом столбце
Результат:
array([1, 2, 3])

>>>a.min(axis=1)  # Наименьшее число в каждой строке
Результат:
array([1, 4])
```

Индексы, срезы, итерации

Одномерные массивы осуществляют операции индексирования, срезов и итераций очень схожим образом с обычными списками и другими последовательностями Python (разве что удалять с помощью срезов нельзя).

```
>>>a = np.arange(10) ** 3 # Создание массива кубов
>>>a
Результат:
array([  0,   1,   8,  27,  64, 125, 216, 343, 512, 729])
```

```
>>>a[1]    # Вывод элемента с номером один( нумерация начинается с нуля)
Результат: 1

>>>a[3:7]    # Срез вывод элементов с номера 3 по 7
Результат:
array([ 27,  64, 125, 216])

>>>a[3:7] = 8
>>>a
Результат:
array([  0,   1,   8,   8,   8,   8,   8, 343, 512, 729])

>>> a[::-1]
Результат:
array([729, 512, 343,   8,   8,   8,   8,   8,   1,   0])

>>> del a[4:6]
Результат:
Traceback (most recent call last):
  File "<input>", line 1, in <module> ValueError: cannot delete array elements

>>>for i in a:
...     print(i ** (1/3))
...
Результат:
0.0
1.0
2.0
2.0
2.0
2.0
2.0
7.0
8.0
9.0
```

У многомерных массивов на каждую ось приходится один индекс. Индексы передаются в виде последовательности чисел, разделенных запятыми (т.е. кортежами):

```
>>> b = np.array([[  0, 1, 2, 3],
[10, 11, 12, 13],
[20, 21, 22, 23],
[30, 31, 32, 33],
[40, 41, 42, 43]])
...
```

```
>>> b[2,3]  # Вторая строка, третий столбец
Результат:  23

>>> b[(2,3)]
Результат:  23

>>> b[2][3]     # Можно и так
Результат:   23

>>> b[:,2]     # Третий столбец
Результат:
        array([ 2, 12, 22, 32, 42])

>>> b[:2]     # Первые две строки
Результат:
        array([[ 0,  1,  2,  3],
        [10, 11, 12, 13]])

>>> b[1:3, : : ]     # Вторая и третья строки
Результат:
        array([[10, 11, 12, 13],
                [20, 21, 22, 23]])
```

Когда индексов меньше, чем осей, отсутствующие индексы предполагаются дополненными с помощью срезов:

```
>>> b[-1]     # Последняя строка. Эквивалентно b[-1,:]
Результат:
        array([40, 41, 42, 43])
```

b[i] можно читать как b[i, <столько символов ':', сколько нужно>]. В NumPy это также может быть записано с помощью точек, как b[i, ...].
Например, если x имеет ранг 5 (то есть у него 5 осей), тогда
x[1, 2, ...] эквивалентно x[1, 2, :, :, :],
x[... , 3] то же самое, что x[:, :, :, :, 3] и
x[4, ... , 5, :] это x[4, :, :, 5, :].

```
>>> a = np.array(([[0, 1, 2], [10, 12, 13]], [[100, 101, 102], [110, 112, 113]]))
>>>a.shape
Результат:
         (2, 2, 3)

>>> a[1, ...]    # то же, что a[1, : , :] или a[1]
Результат:
        array([[100, 101, 102],
                [110, 112, 113]])
```

```
>>> c[... ,2]    # то же, что a[: , : ,2]
```
Результат:
```
array([[  2,  13],
       [102, 113]])
```

Итерирование многомерных массивов начинается с первой оси:
```
>>>for row in a:
...     print(row)
...
```
Результат:
```
[[ 0  1  2]
[10 12 13]]
[[100 101 102]
[110 112 113]]
```

Однако, если нужно перебрать поэлементно весь массив, как если бы он был одномерным, для этого можно использовать атрибут flat:

```
>>>for el in a.flat:
...     print(el)
...
```
Результат:
```
0
1
2
10
12
13
100
101
102
110
112
113
```

Манипуляции с формой

У массива есть форма (shape), определяемая числом элементов вдоль каждой оси:

```
>>> a
```
Результат:
```
array([[[  0,   1,   2],
        [ 10,  12,  13]],
        [[100, 101, 102],
        [110, 112, 113]]])
```

```
>>> a.shape
Результат:
(2, 2, 3)
```

Форма массива может быть изменена с помощью различных команд:

```
>>> a.ravel()    # Делает массив плоским
Результат:
array([  0,   1,   2,  10,  12,  13, 100, 101, 102, 110, 112, 113])

>>> a.shape = (6, 2)    # Изменение формы
>>> a
Результат:
array([[  0,   1],
       [  2,  10],
       [ 12,  13],
       [100, 101],
       [102, 110],
       [112, 113]])

>>> a.transpose()    # Транспонирование
Результат:
       array([[  0,   2,  12, 100, 102, 112],
              [  1,  10,  13, 101, 110, 113]])

>>> a.reshape((3, 4))    # Изменение формы
Результат:
       array([[  0,   1,   2,  10],
              [ 12,  13, 100, 101],
              [102, 110, 112, 113]])
```

Порядок элементов в массиве в результате функции **ravel()** соответствует обычному "С-стилю", то есть, чем правее индекс, тем он "быстрее изменяется": за элементом a[0,0] следует a[0,1]. Если одна форма массива была изменена на другую, массив переформировывается также в "С-стиле". Функции **ravel() и reshape()** могут работать (при использовании дополнительного аргумента) в FORTRAN-стиле, в котором быстрее изменяется более левый индекс.

```
>>>a
Результат:
array([[  0,   1],
       [  2,  10],
       [ 12,  13],
       [100, 101],
       [102, 110],
       [112, 113]])
```

```
>>>a.reshape((3, 4), order='F')
Результат:
array([[0, 100, 1, 101],
       [2, 102,  10, 110],
       [12, 112, 13, 113]])
```

Метод reshape() возвращает ее аргумент с измененной формой, в то время как метод resize() изменяет сам массив:

```
>>>a.resize((2, 6))
>>>a
Результат:
array([[  0,   1,   2,  10,  12,  13],
       [100, 101, 102, 110, 112, 113]])
```

Если при операции такой перестройки один из аргументов задается как -1, то он автоматически рассчитывается в соответствии с остальными заданными:

```
>>> a.reshape((3, -1))
Результат:
array([[  0,   1,   2,  10],
        [ 12,  13, 100, 101],
       [102, 110, 112, 113]])
```

Объединение массивов

Несколько массивов могут быть объединены вместе вдоль разных осей с помощью функций **hstack и vstack.**
hstack() объединяет массивы по первым осям, vstack() — по последним:

```
>>> a = np.array([[1, 2], [3, 4]])
>>> b = np.array([[5, 6], [7, 8]])
>>>np.vstack((a, b))
Результат:
array([[1, 2],
        [3, 4],
       [5, 6],
       [7, 8]])
>>>np.hstack((a, b))
Результат:
array([[1, 2, 5, 6],
       [3, 4, 7, 8]])
```

Функция **column_stack()** объединяет одномерные массивы в качестве столбцов двумерного массива:

```
>>> np.column_stack((a, b))
Результат:
array([[1, 2, 5, 6],
       [3, 4, 7, 8]])
```

Аналогично для строк имеется функция **row_stack().**

```
>>> np.row_stack((a, b))
```

Результат:

```
array([[1, 2],
       [3, 4],
       [5, 6],
       [7, 8]])
```

Разбиение массива

Используя **hsplit()** возможно разбить массив вдоль горизонтальной оси, указав либо число возвращаемых массивов одинаковой формы, либо номера столбцов, после которых массив разрезается "ножницами":

```
>>> a = np.arange(12).reshape((2, 6))
>>>a
```

Результат:

```
array([[ 0,  1,  2,  3,  4,  5],
       [ 6,  7,  8,  9, 10, 11]])

>>> np.hsplit(a, 3)    # Разбить на 3 части
```

Результат:

```
[array([[0, 1], [6, 7]]),
 array([[2, 3], [8, 9]]),
 array([[ 4,  5], [10, 11]])]

>>> np.hsplit(a, (3, 4))    # Разрезать a после третьего и четвёртого столбца
```

Результат:

```
[array([[0, 1, 2], [6, 7, 8]]),
 array([[3], [9]]),
 array([[ 4,  5], [10, 11]])]
```

Функция **vsplit()** разбивает массив вдоль вертикальной оси, а array_split() позволяет указать оси, вдоль которых произойдет разбиение.

Копии и представления

При работе с массивами, их данные иногда необходимо копировать в другой массив, а иногда нет. Это часто является источником ошибок. Возможно 3 случая:

Копии не создаются

Простое присваивание не создает ни копии массива, ни копии его данных:

```
>>> a = np.arange(12)
>>> b = a    # Нового объекта создано не было
>>> b is a   # a и b это два имени для одного и того же объекта ndarray
```

Результат:

```
True
```

```
>>> b.shape = (3,4)    # изменит форму a
>>> a.shape
Результат:
(3, 4)
```

Python передает изменяемые объекты как ссылки, поэтому вызовы функций также не создают копий.

Представление или поверхностная копия

Разные объекты массивов могут использовать одни и те же данные. Метод view() создает новый объект массива, являющийся представлением тех же данных. >>> c = a.view()

```
>>>c is a
Результат:
False

>>> c.base is a  # c это представление данных, принадлежащих a
Результат:
True

>>>c.flags.owndata
Результат:
False

>>>
>>> c.shape = (2,6)  # форма a не поменяется
>>> a.shape
Результат:
 (3, 4)

>>> c[0,4] = 1234  # данные a изменятся
>>> a
Результат:
array([[  0, 1, 2, 3],
      [1234, 5, 6, 7],
[ 8, 9, 10, 11]])
```

Срез массива это представление:

```
>>> s = a[:,1:3]
>>>s[:] = 10
>>>a
Результат:
array([[  0, 10, 10, 3],
          [1234, 10, 10, 7],
          [ 8, 10, 10, 11]])
```

Глубокая копия

Метод **copy()** создаст настоящую копию массива и его данных: >>> d = a.copy() # создается новый объект массива с новыми данными

```
>>> d is a
Результат:
False

>>> d.base is a  # d не имеет ничего общего с a
Результат:
False

>>>d[0, 0] = 9999
>>>a
Результат:
array([[  0, 10, 10, 3],
        [1234, 10, 10, 7],
        [ 8, 10, 10, 11]])
```

Математика многочленов

NumPy предоставляет методы для работы с полиномами. Передавая список корней, можно получить коэффициенты уравнения:

```
>>> np.poly([-1, 1, 1, 10])
Результат:
array([ 1, -11,   9,  11, -10])
```

Здесь массив возвращает коэффициенты, соответствующие уравнению:

$$\mathbf{x^4-11x^3+9x^2+11x-10.}$$

Может быть произведена и обратная операция: передавая список коэффициентов, функция **root** вернет все корни многочлена:

```
>>>np.roots([1, 4, -2, 3])
Результат:
array([-4.57974010+0.j,
        0.28987005+0.75566815j,
        0.28987005-0.75566815j])
```

Заметим, что в этом уравнении x^3+4x^2-2x+3 два корня мнимые.

Коэффициенты многочлена могут быть интегрированы. Рассмотрим интегрирование x^3+x^2+x+1 в $x^4/4+x^3/3+x^2/2+x+C$. Обычно константа C равна нулю:

```
>>>np.polyint([1, 1, 1, 1])
Результат:
array([ 0.25,  0.33333333,  0.5,  1.,  0. ])
```

Аналогично могут быть взяты производные:
>>> np.polyder([1./4., 1./3., 1./2., 1., 0.])
Результат:
array([1., 1., 1., 1.])

Функции **polyadd, polysub, polymul и polydiv** также поддерживают суммирование, вычитание, умножение и деление коэффициентов многочлена, соответственно. Функция polyval подставляет в многочлен заданное значение. Рассмотрим многочлен x3−2x2+2 при x = 4:

>>>np.polyval([1, -2, 0, 2], 4)
Результат: 34

В заключение, функция **polyfit** может быть использована для подбора (интерполяции) многочлена заданного порядка к набору значений:

>>> x = [1, 2, 3, 4, 5, 6, 7, 8]
>>> y = [0, 2, 1, 3, 7, 10, 11, 19]
>>> np.polyfit(x, y, 2)
Результат:
array([0.375, -0.88690476, 1.05357143])

Возвращаемый массив – это список коэффициентов многочлена. Более утонченные интерполяционные функции могут быть найдены в SciPy.

Статистика

В придачу к функциям mean, var и std, NumPy предоставляет еще некоторые методы для работы со статистическими данными в массивах.
Медиана может быть найдена так:

>>> a = np.array([1, 4, 3, 8, 9, 2, 3], float)
>>>np.median(a)
Результат: 3.0

Коэффициент корреляции для некоторых переменных наблюдается несколько раз и может быть найден из массивов вида: [[x1, x2, ...], [y1, y2, ...], [z1, z2, ...], ...], где x, y, z - это разные квантовые наблюдаемые, и номера указывают количество «наблюдений»:

>>> a = np.array([[1, 2, 1, 3], [5, 3, 1, 8]], float)
>>> c = np.corrcoef(a)
>>>c
Результат:
array([[1., 0.72870505],
        [0.72870505, 1.]])

Имеем возвращаемый массив c[i, j] который хранит корреляционный коэффициент для i-тых и j-тых квантовых наблюдаемых.
Аналогично, ковариационный момент может быть найден:

```
>>>np.cov(a)
Результат:
        array([[ 0.91666667,  2.08333333],
               [ 2.08333333,  8.91666667]])
```

NumPy: случайные числа

Важная часть каждой симуляции – это способность генерировать случайные числа. Для генерации NumPy использует особенный алгоритм, который имеет название **Mersenne Twister**.
Задать порождающий элемент последовательности случайных чисел можно так:
>>> np.random.seed(293423)
Seed – это целое число. Каждая программа, которая запускается с одинаковым seed`ом, будет генерировать одинаковую последовательность чисел каждый раз. Это может быть полезно для отладки, но вообще нам не нужно задавать seed, на самом деле, когда мы запускаем программу несколько раз, мы хотим получать каждый раз разную последовательность чисел. Если эта команда не будет выполнена, то NumPy автоматически выбирает случайный seed (базирующийся на времени), который является разным при каждом запуске программы.

Инициализация генератора случайных чисел

seed(число) - инициализация генератора.

```
>>> np.random.seed(1000)
>>> np.random.random(10)
Результат:
array([ 0.65358959,  0.11500694,  0.95028286,  0.4821914 ,  0.87247454,
    0.21233268,  0.04070962,  0.39719446,  0.2331322 ,  0.84174072])

>>> np.random.seed(1000)
>>> np.random.random(10)
Результат:
array([ 0.65358959,  0.11500694,  0.95028286,  0.4821914 ,  0.87247454,
0.21233268,  0.04070962,  0.39719446,  0.2331322 ,  0.84174072])
```

get_state и set_state - возвращают и устанавливают состояние генератора.

```
>>>np.random.seed(1000)
>>>state = np.random.get_state()
>>>np.random.random(10)
Результат:
array([ 0.65358959,  0.11500694,  0.95028286,  0.4821914 ,  0.87247454,
    0.21233268,  0.04070962,  0.39719446,  0.2331322 ,  0.84174072])
```

```
>>> np.random.set_state(state)
>>>np.random.random(10)
```
Результат:
```
array([ 0.65358959,  0.11500694,  0.95028286,  0.4821914 ,  0.87247454,
0.21233268,  0.04070962,  0.39719446,  0.2331322 ,  0.84174072])
```

Массив случайных чисел из полуинтервала [0.0, 1.0) может быть сгенерирован так:

```
>>> np.random.rand(5)
```
Результат:
```
array([ 0.40783762,  0.7550402 ,  0.00919317,  0.01713451,  0.95299583])
```

Функция **rand** может быть использована для генерации двумерных массивов, или можно использовать функцию **reshape**:

```
>>>np.random.rand(2,3)
```
Результат:
```
array([[ 0.50431753,  0.48272463,  0.45811345],
[ 0.18209476,  0.48631022,  0.49590404]])
```

```
>>>np.random.rand(6).reshape((2,3))
```
Результат:
```
array([[ 0.72915152,  0.59423848,  0.25644881],
[ 0.75965311,  0.52151819,  0.60084796]])
```

Для генерации единичного случайного числа на интервале [0.0, 1.0):

```
>>> np.random.random()
```
Результат:
```
0.70110427435769551
```

Создавать списки, используя встроенный модуль random, а затем преобразовывать их в numpy.array:

```
>>> import numpy as np
>>>import random
>>>np.array([random.random() for i in range(10)])
```
Результат:
```
array([ 0.99538667,  0.16860511,  0.78952804,  0.09676316,  0.86110208,
0.89674666,  0.56401347,  0.63431468,  0.51110935,  0.64944844])
```

Другой способ – numpy.random

Для создания массивов со случайными элементами служит модуль numpy.random.

```
>>>importnumpyasnp    #Импортировать numpy и писать np.random
>>>np.random
Результат:
<module 'numpy.random' from '/usr/local/lib/python3.4/dist-
packages/numpy/random/__init__.py'>

>>> import numpy.random as rand    #Можно и присвоить отдельное имя. rand
>>>rand
Результат:
<module 'numpy.random' from '/usr/local/lib/python3.4/dist-
packages/numpy/random/__init__.py'>
```

Создание массивов

Самый простой способ задать массив со случайными элементами – использовать функцию **sample** (или random, или random_sample, или ranf - это всё одна и та же функция).

```
>>>np.random.sample()
Результат:   0.6336371838734877


>>>np.random.sample(3)
Результат:
array([ 0.53478558,  0.1441317 ,  0.15711313])

>>>np.random.sample((2, 3))
Результат:
array([[ 0.12915769,  0.09448946,  0.58778985],
[ 0.45488207,  0.19335243,  0.22129977]])
```

Без аргументов возвращает просто число в промежутке [0, 1), с одним целым числом – одномерный массив, с кортежем – массив с размерами, указанными в кортеже (все числа – из промежутка [0, 1)).
С помощью функции **randint** или **random_integers** можно создать массив из целых чисел. Аргументы: low, high, size: от какого, до какого числа (randint не включает в себя это число, а random_integers включает), и size – размеры массива.

```
>>>np.random.randint(0, 3, 10)
Результат:
array([0, 2, 0, 1, 1, 0, 2, 2, 2, 0])

>>> np.random.random_integers(0, 3, 10)
Результат:
array([2, 2, 3, 3, 1, 1, 0, 2, 3, 2])
```

```
>>>np.random.randint(0, 3, (2, 10))
Результат:
array([[0, 1, 2, 0, 0, 0, 1, 1, 1, 2],
[0, 0, 2, 2, 2, 0, 1, 2, 2, 1]])
```

Можно генерировать числа согласно различными распределениями(Гаусса, Парето и другие). Чаще всего нужно равномерное распределение, которое можно получить с помощь функции uniform.

```
>>>np.random.uniform(2, 8, (2, 10))
Результат:
array([[ 3.1517914 ,  3.10313483,  2.84007134,  3.21556436,  4.64531786,
2.99232714,  7.03064897,  4.38691765,  5.27488548,  2.63472454],
[ 6.39470358,  5.63084131,  4.69996748,  7.07260546,  7.44340813,
4.10722203,  7.52956646,  4.8596943 ,  3.97923973,  5.64505363]])
```

NumPy также включает генераторы для других распределений, таких как: Бета, биномиальное, хи-квадрат, Дирихле, экспоненциальное, Фишера, Гамма, геометрическое, Гамбала, гипергеометрическое, Лапласа, логистическое, логнормальное, логарифмическое, мультиномиальное, многомерное нормальное, отрицательное биномиальное, нецентральное хи-квадрат, нецентральное Фишера, нормальное (Гаусса), Парето, Пуассона, степенное, Рэлея, Коши, Стьюдента, треугольное, Фон-Миса, Вальда, Вейбулла и Ципфа.

Рассмотрим два примера.
Для генерации из дискретного распределения Пуассона при $\lambda = 6.0$,

```
>>> np.random.poisson(6.0)
Результат:  5
```

Для генерации числа из нормального распределения (Гаусса) при среднем значении $\mu = 1.5$ и стандартной девиации $\sigma = 4.0$:

```
>>> np.random.normal(1.5, 4.0)
Результат:
0.83636555041094318
```

Для получении числа из нормального распределения ($\mu = 0$, $\sigma = 1$), без указания аргументов:

```
>>> np.random.normal()
Результат:
0.27548716940682932
```

Для генерации нескольких значений используем аргумент size:

```
>>>np.random.normal(size=5)
Результат:
array([-1.67215088,  0.65813053, -0.70150614,  0.91452499,  0.71440557])
```

Выбор и перемешивание

Перемешать NumPy массив можно с помощью функции **shuffle**:

```
>>> a = np.arange(10)
>>>a
Результат:
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

>>>np.random.shuffle(a)
>>>a
Результат:
array([2, 8, 7, 3, 5, 0, 4, 9, 1, 6])
```

Также можно перемешать массив с помощью функции **permutation** (она, в отличие от shuffle, возвращает перемешанный массив). Также она, вызванная с одним аргументом (целым числом), возвращает перемешанную последовательность от 0 до N.

```
>>>np.random.permutation(10)
Результат:
array([1, 2, 3, 8, 7, 9, 4, 6, 5, 0])
```

Сделать случайную выборку из массива можно с помощью функции **choice**. Про неё стоит рассказать подробнее.

numpy.random.choice(a, size=None, replace=True, p=None)
a: одномерный массив или число. Если массив, будет производиться выборка из него. Если число, то выборка будет производиться из np.arange(a).
size : размерности массива. Если None, возвращается одно значение.
replace : если True, то одно значение может выбираться более одного раза.
p: вероятности. Это означает, что элементы можно выбирать с неравными вероятностями. Если не заданы, используется равномерное распределение.

```
>>>a = np.arange(10)
>>>a
Результат:
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

>>>np.random.choice(a, 10, p=[0.5, 0.25, 0.25, 0, 0, 0, 0, 0, 0, 0])
Результат:
array([0, 0, 0, 0, 1, 2, 0, 0, 1, 1])
```

Операторы сравнения и тестирование значений

Булево сравнение может быть использовано для поэлементного сравнения массивов одинаковых длин. Возвращаемое значение это массив булевых True/False значений:

```
>>>a = np.array([1, 3, 0], float)
>>> b = np.array([0, 3, 2], float)
>>> a > b
Результат:
array([ True, False, False], dtype=bool)

>>>a == b
Результат:
array([False,  True, False], dtype=bool)

>>> a <= b
Результат:
array([False,  True,  True], dtype=bool)
```

Результат сравнения может быть сохранен в массиве:

```
>>> c = a > b
>>>c
Результат:
array([ True, False, False], dtype=bool)
```

Массивы могут быть сравнены с одиночным значением:

```
>>> a = np.array([1, 3, 0], float)
>>>a> 2
Результат:
array([False,  True, False], dtype=bool)
```

Операторы any и all могут быть использованы для определения истинны ли хотя бы один или все элементы соответственно:

```
>>> c = np.array([ True, False, False], bool)
>>>any(c)
Результат:   True

>>>all(c)
Результат:  False
```

Комбинированные булевы выражения могут быть применены к массивам по принципу элемент–элемент, используя специальные функции logical_and, logical_or и logical_not:

```
>>> a = np.array([1, 3, 0], float)
>>> np.logical_and(a > 0, a < 3)
Результат:
array([ True, False, False], dtype=bool)

>>> b = np.array([True, False, True], bool)
>>> np.logical_not(b)
Результат:
array([False,  True, False], dtype=bool)

>>> c = np.array([False, True, False], bool)
>>> np.logical_or(b, c)
Результат:
array([ True,  True,  False], dtype=bool)
```

Функция **where** создает новый массив из двух других массивов одинаковых длин, используя булев фильтр для выбора межу двумя элементами. Базовый синтаксис: **where(boolarray, truearray, falsearray**):

```
>>> a = np.array([1, 3, 0], float)
>>>np.where(a != 0, 1 / a, a)
Результат:
array([ 1.,  0.33333333,  0. ])
```

С функцией where так же может быть реализовано «массовое сравнение»:

```
>>>np.where(a> 0, 3, 2)
Результат:
array([3, 3, 2])
```

Некоторые функции дают возможность тестировать значения в массиве. Функция **nonzero** возвращает кортеж индексов ненулевых значений. Количество элементов в кортеже равно количеству осей в массиве:

```
>>> a = np.array([[0, 1], [3, 0]], float)
>>>a.nonzero()
Результат:
(array([0, 1]), array([1, 0]))
```

Также можно проверить значения на конечность и NaN (not a number):

```
>>> a = np.array([1, np.NaN, np.Inf], float)
>>>a
Результат:
array([  1.,  NaN,  Inf])
```

```
>>>np.isnan(a)
Результат:
array([False,  True, False], dtype=bool)

>>>np.isfinite(a)
Результат:
array([ True, False, False], dtype=bool)
```

Хотя здесь мы использовали константы numpy чтобы добавить значения NaN и бесконечность, они могут быть результатами применения стандартных математических операций.

Выбор элементов массива и манипуляция с ними

Элементы массива можно получить, используя операцию доступа по индексу. Однако, в отличие от списков, массивы также позволяют делать выбор элементов, используя другие массивы. Это значит, что мы можем использовать массив для фильтрации специфических подмножеств элементов других массивов.
Булевы массивы могут быть использованы как массивы для фильтрации:

```
>>> a = np.array([1, np.NaN, np.Inf], float)
>>>a
Результат:
array([  1.,  NaN,  Inf])

>>>np.isnan(a)
Результат:
array([False,  True, False], dtype=bool)

>>>np.isfinite(a)
Результат:
array([ True, False, False], dtype=bool)
```

Стоит заметить, что когда мы передаем булев массив a >= 6 как индекс для операции доступа по индексу массива a, возвращаемый массив будет хранить только True значения. Также мы можем записать массив для фильтрации в переменную:

```
>>>a = np.array([[6, 4], [5, 9]], float)
>>>sel = (a>= 6)
>>>a[sel]
Результат:
array([ 6.,  9.])
```

Более сложная фильтрация может быть достигнута использованием булевых выражений:

```
>>>a[np.logical_and(a > 5, a < 9)]
>>> array([ 6.])
```

В придачу к булеву выбору также можно использовать целочисленные массивы. В этом случае целочисленный массив хранит индексы элементов, которые будут взяты из массива. Рассмотрим следующий одномерный пример:

```
>>>a = np.array([2, 4, 6, 8], float)
>>>b = np.array([0, 0, 1, 3, 2, 1], int)
>>> a[b]
```

Результат:

```
array([ 2.,  2.,  4.,  8.,  6.,  4.])
```

Иными словами, когда мы используем b для получения элементов из a, мы берем 0-й, 0-й, 1-й, 3-й, 2-й и 1-й элементы a в этом порядке. Списки также могут быть использованы как массивы для фильтрации:

```
>>>a = np.array([2, 4, 6, 8], float)
>>>a[[0, 0, 1, 3, 2, 1]]
```

Результат:

```
array([ 2.,  2.,  4.,  8.,  6.,  4.])
```

Для многомерных массивов нам необходимо передать несколько одномерных целочисленных массивов в оператор доступа индексу для каждой оси. Потом каждый из массивов проходит такую последовательность: первый элемент соответствует индексу строки, который является первым элементом массива b, второй элемент соответствует индексу столбца, который является первым элементом массива c и так далее.
Пример:

```
>>> a = np.array([[1, 4], [9, 16]], float)
>>> b = np.array([0, 0, 1, 1, 0], int)
>>> c = np.array([0, 1, 1, 1, 1], int)
>>>a[b,c]
```

Результат:

```
array([  1.,   4.,  16.,  16.,   4.])
```

Специальная функция **take** доступна для выполнения выборки с целочисленными массивами. Это работает также как и использования оператора взятия по индексу:

```
>>> a = np.array([2, 4, 6, 8], float)
>>> b = np.array([0, 0, 1, 3, 2, 1], int)
>>> a.take(b)
```

Результат:

```
        array([ 2.,  2.,  4.,  8.,  6.,  4.])
```

Функция take также предоставляет аргумент axis (ось) для взятия подсекции многомерного массива вдоль какой-либо оси*.

* Прим. переводчика: по строкам или столбцам (для двумерных массивов)

```
>>>a = np.array([[0, 1], [2, 3]], float)
>>> b = np.array([0, 0, 1], int)
>>>a.take(b, axis=0)
```
Результат:
```
array([[ 0.,  1.],
       [ 0.,  1.],
       [ 2.,  3.]])
```

```
>>>a.take(b, axis=1)
```
Результат:
```
array([[ 0.,  0.,  1.],
       [ 2.,  2.,  3.]])
```

В противоположность к функции take есть функция **put**, которая будет брать значения из исходного массива и записывать их на специфические индексы в другом put-массиве.

```
>>> a = np.array([0, 1, 2, 3, 4, 5], float)
>>> b = np.array([9, 8, 7], float)
>>>a.put([0, 3], b)
>>> a
```
Результат:
```
array([ 9.,  1.,  2.,  8.,  4.,  5.])
```

Заметим, что значение 7 из исходного массива b не было использовано, так как указаны только 2 индекса [0, 3]. Исходный массив будет повторен, если необходимо в случае несоответствия длин:

```
>>> a = np.array([0, 1, 2, 3, 4, 5], float)
>>>a.put([0, 3], 5)
>>> a
```
Результат:
```
array([ 5.,  1.,  2.,  5.,  4.,  5.])
```

Практическое задание

1. Создать вектор (одномерный массив) размера 10, заполненный нулями
2. Импортировать NumPy под именем np
3. Создать вектор размера 10, заполненный единицами
4. Создать вектор размера 10, заполненный числом 2.5
5. Напечатать версию и конфигурацию
6. Создать вектор размера 10, заполненный нулями, но пятый элемент равен 1
7. Создать вектор со значениями от 10 до 49
8. Развернуть вектор (первый становится последним)
9. Создать матрицу (двумерный массив) 3 x 3 со значениями от 0 до 8
10. Найти индексы ненулевых элементов в [1,2,0,0,4,0]
11. Создать 3 x 3 единичную матрицу
12. Создать массив 3 x 3 x 3 со случайными значениями
13. Создать массив 10 x 10 со случайными значениями, найти минимум и максимум
14. Создать случайный вектор размера 30 и найти среднее значение всех элементов
15. Создать матрицу с 0 внутри, и 1 на границах
16. Создать 5 x 5 матрицу с 1,2,3,4 под диагональю
17. Создать 8 x 8 матрицу и заполнить её в шахматном порядке
18. Отсортировать вектор
19. Проверить, одинаковы ли 2 numpy массива
20. Заменить максимальный элемент на ноль
21. Найти минимальное и максимальное значение, принимаемое каждым числовым типом numpy
22. Преобразовать массив из float в int
23. Отнять среднее из каждой строки в матрице
24. Отсортировать матрицу по n-ому столбцу
25. Дан четырехмерный массив, посчитать сумму по последним двум осям
26. Найти диагональные элементы произведения матриц
27. Поменять 2 строки в матрице
28. Посчитать ранг матрицы
29. Найти наиболее частое значение в массиве

Ресурсы:

- M. Scott Shell – An introduction to Numpy and Scipy
https://sites.engineering.ucsb.edu/~shell/che210d/numpy.pdf

- NumPy в Python – перевод
https://habr.com/ru/post/352678/

- Фундаментальный пакет для научных вычислений с Python
https://numpy.org/

- Сообщество NumPy и SciPy, онлайн руководство
https://docs.scipy.org/doc

- NumPy — Краткое руководство
https://coderlessons.com/tutorials/python-technologies/uchitsia-numpy/numpy-kratkoe-rukovodstvo

- 5 ключевых библиотек и пакетов для анализа данных на Python
https://techrocks.ru/2018/07/22/5-key-libraries-and-packets-for-data-analysis-in-python/

- Изучение NumPy с визуальными примерами для начинающих
https://python-scripts.com/numpy

- Практический Python 3 для начинающих
https://pythonworld.ru/numpy/1.html